Local search and a construction scan for covering-design and lottery-covering tables

Wynand van Staden
Independent researcher, South Africa
vanstadenw24@gmail.com
ORCID: 0009-0009-9884-8547

27 October 2026

Abstract

A covering design $C(v, k, t)$ is a family of k-subsets (blocks) of a v-set containing every t-subset; a lottery covering $C(v, k, t, m)$ only requires every m-subset to meet some block in at least t points. We describe the methods behind about a thousand improvements of the public tables of both. For covering designs, a decremental tabu search with clause weighting, as in set-cover local search, keeps exact move scores incrementally. For lottery coverings, the same search scores moves at the level of t-subsets by aggregating m-subset weights, so that a move is scored with as many lookups as for covering designs (updating after a move remains costly); we also describe engineering for covers with up to $2 \cdot 10^9$ m-subsets. Finally, a dynamic-programming scan applies standard recursive constructions to the whole live table, and the explicit products are then reduced by the search. We evaluated the searches in a test whose protocol and sample were fixed in advance: 300 random covering-design entries and 300 random lottery entries not held by us, two minutes per run. The weighted search improved 14% of the covering-design entries and the same search without weighting 8%. On lottery entries the methods improved 1.3–4.3%, and the t-level search did better without weighting than with it. On 25 October 2026 the Covering Repository held 998 of our coverings: 401 covering designs and 65 lottery coverings found by search, and 532 constructed coverings.

1 Introduction

Let V be a set of v points. A (v, k, t, m) *lottery covering* is a family $\mathcal{B}$ of k-subsets of V, the *blocks*, such that every m-subset $M \subseteq V$ satisfies $|B \cap M| \geq t$ for some $B \in \mathcal{B}$. The smallest size of such a family is $C(v, k, t, m)$; we also write $C(v, k, t, m)$ for a covering. The lottery reading is that if the m drawn numbers lie among the v chosen ones, some ticket matches at least t of them. These objects are also called lotto designs, with notation $L(n, k, p, t)$ for $n = v$, $p = m$ [2, 10, 9], and general covering designs [14]. For $m = t$ the condition says that every t-subset lies in a block, and $C(v, k, t) = C(v, k, t, t)$ is the covering number of a covering design [13, 8], bounded below by the Schönheim bound $\lceil \frac{v}{k} \lceil \frac{v-1}{k-1} \cdots \lceil \frac{v-t+1}{k-t+1} \rceil \cdots \rceil \rceil$ [20]. Finding $C(v, k, t)$ is a unicost set-cover problem with one element per t-subset and one set per k-subset.

Since exact values are known only in small cases, tables record the smallest known coverings. The La Jolla Covering Repository recorded covering designs for $v < 100$, $k \leq 25$, $t \leq 8$ until March 2026, and its data are archived [7]. The Covering Repository [3] keeps covering designs and lottery coverings, accepts uploads and checks each one with its own validator.

Related work. Upper bounds come from algebraic and recursive constructions [6, 14], greedy methods [6], simulated annealing [16], tabu search with fast incremental evaluation [5], general upper-bound constructions for $C(v, k, t, m)$ [1], and exact or constraint-based methods for small cases [12, 4]; lotto-design tables combine many of these [10]. Our search borrows from local search for satisfiability and set cover: clause weighting, which raises the weight of every unsatisfied clause so that the search is pushed out of local minima, as in the breakout method [15] and dynamic local search for SAT [19]; and row-weighting local search for unicost set cover, RWLS [17] and NuSC [11], which also search decrementally (find a cover, delete a set, repair).

Contributions.

- A decremental, clause-weighted local search for covering designs that combines the neighbourhood of covering-design search [16, 5] with the weighting and decremental scheme of set-cover local search [17], with exact incremental scores (Section 3.1).
- A systematic scan that applies known constructions [6, 14] to the whole live table by dynamic programming, including the searches' own fresh records, followed by explicit construction, certification and reduction of the products by the search (Section 5).
- An evaluation on a random sample of table entries, with a protocol fixed in advance, comparing the search with weighting switched off and with simulated annealing (Section 7).
- An extension to lottery coverings that scores moves by t-subset aggregates of m-subset weights, which are the make and break scores of SAT local search on the monotone formula of Lemma 1 (Section 3.2), and engineering for very large covers (Section 4). On lottery coverings the evaluation is mostly negative: all methods improved few entries, and the weighting did not help the t-level search.

Section 6 describes verification, Section 7 evaluates the searches on a random sample of table entries, Section 8 reports the published coverings by method, Section 9 says where the methods do not work, and Section 10 sums up. Code, record lists and construction recipes are public [21].

2 Notation

For a t-subset T and a family $\mathcal{B}$, let $\mathrm{ct}(T) = |\{B \in \mathcal{B} : T \subseteq B\}|$. For an m-subset M, let $\mathrm{cnt}(M) = |\{T \subseteq M : |T| = t,\ \mathrm{ct}(T) > 0\}|$, the number of covered t-subsets of M.

Lemma 1. *An m-subset M is covered (that is, $|B \cap M| \geq t$ for some block B) if and only if* $\mathrm{cnt}(M) > 0$.

Proof. If $|B \cap M| \geq t$, any t-subset T of $B \cap M$ lies in B and in M, so $\mathrm{cnt}(M) > 0$. Conversely, if a t-subset $T \subseteq M$ lies in a block B, then $|B \cap M| \geq t$. □

A *move* replaces a point x of a block B by a point $y \notin B$. Only the $\binom{k-1}{t-1}$ t-subsets of B that contain x can lose a block, and only the t-subsets of $B - x + y$ that contain y can gain one. A t-subset *flips* when it changes between covered ($\mathrm{ct} > 0$) and uncovered.

3 Search

3.1 Covering designs

The search continues two lines of work. From local search for set cover it takes clause weighting, tabu and decremental search as in RWLS [17]: the weights of uncovered elements grow after each step, a tabu rule prevents immediate reversals, and a cover is shrunk by deleting a set and repairing. From local search for covering designs [16, 5] it takes the representation and the neighbourhood: a fixed number of blocks, and moves that replace one point of a block. The combination is needed because the sets of the set-cover formulation, all $\binom{v}{k}$ k-subsets, cannot be listed, so moves cannot add or remove listed sets as in RWLS.

The state is a family of b blocks, a weight $w(T) \geq 1$ on every t-subset (initially 1), and $\mathrm{ct}(T)$. The cost is the total weight of uncovered t-subsets. One step is:

1. *Focus.* Pick an uncovered t-subset T^*: the heaviest of 4 sampled with replacement, or with probability 0.05 a uniformly random one.

2. *Candidates.* For every block B with $|B \cap T^*| = t - 1$, consider bringing in the missing point y of T^* and removing some $x \in B \setminus T^*$. If no block qualifies, the step is skipped.

3. *Choice.* Take the candidate with the best weighted gain (weight newly covered minus weight newly uncovered), ties broken uniformly at random. A point removed from a block may not re-enter that block during the next 6 to 9 steps (tabu tenure 7 plus a random 0–3, counted from the current step); if every candidate is tabu, a random candidate is taken.

4. *Weighting.* Add 1 to $w(T)$ for every uncovered T.

Step 4 applies the clause-weighting rule after every move rather than only at local minima. There is no restart and no weight decay; a covered t-subset keeps its weight.

Decremental mode. The search starts from a full covering, usually the current table entry. Whenever the family is a covering it is saved, the block that covers the fewest t-subsets alone is deleted (ties to the lowest index), the weights of the newly uncovered t-subsets are reset to 1, and the search repairs. Runs stop at a time limit or a target size.

Exact incremental scores. Our first implementation scored each candidate by enumerating the affected t-subsets; the second keeps the scores incrementally. For a block B and $x \in B$, let $\mathrm{brk}(B, x)$ be the total weight of the t-subsets T with $x \in T \subseteq B$ and $\mathrm{ct}(T) = 1$; this is the loss of removing x. Weights change only on uncovered t-subsets, which never contribute to brk, so brk changes only when some $\mathrm{ct}(T)$ moves between 0, 1 and 2: then $\mathrm{brk}(B', x)$ changes by $\pm w(T)$ for the sole owner B' of T and each $x \in T$ (gain on $0 \to 1$ and $2 \to 1$, loss on $1 \to 0$ and $1 \to 2$). To find the owner in $O(1)$ we use a variant of the critical-variable bookkeeping found in implementations of SAT local search (for example the code accompanying probSAT [18]), which track, for each clause with one true literal, the variable that makes it true; we do not claim it as new. Give each block a fixed identifier and let every t-subset store $\sigma(T) = \sum_{B \ni T} \mathrm{id}(B)$, updated whenever a block gains or loses T.

Lemma 2. *If* $\mathrm{ct}(T) = 1$*, the unique block containing* T *has identifier* $\sigma(T)$*.*

Proof. The sum has one term. □

		blocks after 2 min		steps per second		
Entry	Start	recompute	incremental	recompute	incremental	Schönheim
$C(48, 9, 3)$	292	275	**273**	5,916	66,463	224
$C(54, 13, 4)$	746	668	**666**	391	7,112	536
$C(70, 12, 3)$	352	343	**340**	2,027	36,166	257
$C(35, 18, 6)$	223	211	**207**	12	200	121
$C(90, 12, 3)$	704	683	**679**	1,472	25,745	548

Table 1: Scores recomputed per candidate versus kept incrementally; two-minute runs from the same coverings, one seed each.

When a block is deleted, the last block takes its identifier and the σ values of its t-subsets are corrected. The gain of bringing y into B while removing x is the weight of the uncovered t-subsets $S \cup \{y\}$ with S a $(t-1)$-subset of $B - x$. It is computed per candidate block, not stored: one pass over the $(t-1)$-subsets S of B accumulates the total G and, for each point $p \in B$, the part $g(p)$ from sets $S \ni p$, so that the gain for outgoing x is $G - g(x)$. This costs $O(t\binom{k}{t-1} + k)$ for all k outgoing points together, independently of v. A build with consistency checks recomputes ct, σ and brk by brute force every 5,000 steps.

Table 1 compares the two implementations of the same search from the same start coverings. Incremental scores give 11–18 times more steps per second and better coverings in the same time. The table also gives the Schönheim bound; the records are 22–71% above it.

3.2 Lottery coverings

The direct generalisation puts the weights on m-subsets and scores each move by enumerating the m-subsets it affects. It is correct but slow: in our first tests, with 12 sampled candidate blocks, it removed a block from one of 78 record coverings. Lemma 1 allows scoring at the level of t-subsets instead, with 10–25 times more steps per second (Appendix A). With the larger candidate sample of our batch runs, however, direct scoring improved more random entries than the weighted t-level search (Section 7).

State. We maintain $\mathrm{ct}(T)$ for every t-subset, and $\mathrm{cnt}(M)$ and a weight $w(M)$ for every m-subset, together with the aggregates

$$A(T) = \sum\{w(M) : T \subseteq M,\ \mathrm{cnt}(M) = 0\}, \qquad L(T) = \sum\{w(M) : T \subseteq M,\ \mathrm{cnt}(M) = 1\},$$

and $U(T)$, the number of uncovered m-subsets containing T. For uncovered T, $A(T)$ is the weight that covering T would gain. For covered T, an m-subset counted in $L(T)$ has T as its only covered t-subset, so $L(T)$ is the weight lost if T becomes uncovered; L is read only for covered T. A move $x \to y$ in block B is scored with $\binom{k-1}{t-1}$ terms each:

$$\mathrm{loss} = \sum_{\substack{x \in T \subseteq B \\ \mathrm{ct}(T)=1}} L(T), \qquad \mathrm{gain} = \sum_{\substack{y \in T \subseteq B-x+y \\ \mathrm{ct}(T)=0}} A(T).$$

In SAT terms, Lemma 1 makes a lottery covering a monotone CNF formula with one Boolean variable per t-subset (true when covered) and one clause per m-subset (the disjunction of its t-subsets); $\mathrm{cnt}(M)$ is the number of true literals of a clause, and A and L are the weighted make

and break scores of dynamic local search for SAT [19]. What differs is that the variables are not set freely: a point swap flips a whole bundle of t-subset variables at once, and the instance is far too large to write down. The search therefore sums per-variable make and break scores over the bundle, which is where the estimate below comes from.

Remark 3. This score is an estimate; the bookkeeping applied after a move is exact. It can err in three ways: (a) an m-subset containing two newly covered t-subsets is counted twice in the gain; (b) an m-subset with $\mathrm{cnt}(M) \geq 2$ whose covered t-subsets are all lost is missing from the loss; two lost t-subsets (both contain x) can share $t-1$ points, so their union of $t+1 \leq m$ points can lie in one m-subset; (c) an m-subset with $\mathrm{cnt}(M) = 1$ that loses its covered t-subset but gains another is counted as lost. The score is exact when $m = t$ or when at most one t-subset flips. We have not measured how often the errors change the chosen move.

Updates. When a t-subset flips, $\mathrm{cnt}(M)$ changes by one for each of its $\binom{v-t}{m-t}$ supersets M, and only the changes $0 \leftrightarrow 1$ and $1 \leftrightarrow 2$ of $\mathrm{cnt}(M)$ affect the aggregates. When $\mathrm{cnt}(M)$ goes $0 \to 1$, $w(M)$ leaves A and U decreases on all $\binom{m}{t}$ t-subsets of M, and $w(M)$ joins L; $1 \to 0$ is the reverse; $1 \leftrightarrow 2$ removes or adds $w(M)$ in L. In the first implementation L is kept on all $\binom{m}{t}$ t-subsets of M (only the covered one is ever read); Section 4 keeps it on the covered one only.

Weight bumps in $O(1)$. The clause-weighting step adds the same amount to every uncovered m-subset. We keep a global counter E and, for an uncovered M, a base weight $w_0(M)$ with true weight $w_0(M) + E$. Then $A(T) = A_0(T) + E \cdot U(T)$ with $A_0(T) = \sum w_0(M)$ over the same sets, and a bump adds the bump size to E. When M becomes covered its weight is frozen at $w_0(M)+E$; when it becomes uncovered again its base is set to its weight minus E.

The search. The loop is that of Section 3.1 with these differences. The focus is an uncovered m-subset M^*, the heaviest of 4 sampled. Candidates are blocks meeting M^* in $t-1$ points, with any of the $m-t+1$ points of M^* outside the block as incoming point; 12 candidate blocks are sampled by default and up to 1,000 in our batch runs. A removed point is tabu for the next 4 to 6 steps, there is no random-focus noise, and a step whose candidates are all tabu is skipped. If no block meets M^* in $t-1$ points, a random block swaps a random point outside M^* for a random point of M^*, without tabu or weight bump. On each full covering all weights are reset to 1 and the block with the smallest $\sum_{T \subseteq B,\ \mathrm{ct}(T)=1} L(T)$ is deleted; this counts the m-subsets whose only covered t-subset lies only in B, a lower bound on what B covers alone.

A consistency-check build recomputes ct, cnt, A and L by brute force every 2,000 steps. The implementation needs about 14 bytes per m-subset during search (more at start-up) and refuses $\binom{v}{m} > 3 \cdot 10^8$. The effects of the main settings (candidate sample, stopping rule, seeds and tenures) are listed in Appendix A.

4 Large covers

Constructed covers can have 10^5 blocks or more and $\binom{v}{m}$ near 10^9. A second implementation keeps the search of Section 3.2 and changes the data structures; it accepts $\binom{v}{m} \leq 2 \cdot 10^9$ and also runs covering designs ($m = t$). Its control settings (deletion pool, weight growth, reverts) are in Appendix B.

Version	steps/s	after 10 minutes
first lottery implementation	–	out of memory
large-cover version, parallel setup and deletion pool	≈ 33	stalled near 366,000
+ witness byte, reverts, growing bump	≈ 120	
+ parallel flips, scoring and scan	270–350	$\approx 363{,}000$

Table 2: $C(35, 10, 9, 12)$ (834,451,800 12-subsets) from a constructed covering of 404,385 blocks.

Witness byte. L is now kept only on the covered t-subset of each M with $\mathrm{cnt}(M) = 1$, so a change $1 \leftrightarrow 2$ of $\mathrm{cnt}(M)$ must find that t-subset. Sort M, number the t-subsets $T \subseteq M$ by the colex rank $\lambda(T) \in \{0, \ldots, \binom{m}{t} - 1\}$ of the positions of $M \setminus T$ in the sorted list, and keep one byte $s(M) = \sum \lambda(T) \bmod 256$ over the covered t-subsets T of M.

Lemma 4. *If $\binom{m}{t} \le 256$ and $\mathrm{cnt}(M) = 1$, the covered t-subset of M is the one with $\lambda = s(M)$.*

Proof. The sum has one term $\lambda < \binom{m}{t} \le 256$, so reduction mod 256 leaves it unchanged, and λ determines $M \setminus T$ and hence T. □

A $1 \leftrightarrow 2$ change then costs one update instead of $\binom{m}{t}$ (on $2 \to 1$ the byte is updated before it is read, on $1 \to 2$ after). The program refuses $\binom{m}{t} > 256$.[1] It also refuses parameters where one flip would cost more than 10^8 updates, that is $\binom{v-t}{m-t}\binom{m}{t} > 10^8$.

Parallel updates. A flip enumerates the supersets of T in parallel chunks. Each m-subset is visited by exactly one thread, so per-m-subset state needs no synchronisation; shared t-subset aggregates use atomic additions and the list of uncovered m-subsets a lock. Setup, score rebuilds, the candidate scan and candidate scoring are also parallel above size thresholds; with several threads, runs are not reproducible from the seed.

Table 2 shows the effect on one large cover.

5 A scan over standard constructions

5.1 Rules

The following rules are standard; see [6] for covering designs, [14] for general coverings and [4] for disjoint unions of covering designs as lottery designs. We state them in the form the scan uses. Blocks may be built with fewer than k points; they are completed to k points with arbitrary further points at the end, which can only help. Correspondingly the value for $k > v$ is taken equal to that for $k = v$.

Proposition 5 (Product on a split)**.** *Let $V = V_1 \sqcup V_2$ with $|V_1| = v_1$, $|V_2| = v_2$. Let $\mathcal{D}_i$ be a (v_i, k_i, t_i, m_i) covering on V_i, with $k_1 + k_2 = k$ and $t_1 + t_2 \ge t$. Then $\{B_1 \cup B_2 : B_i \in \mathcal{D}_i\}$ covers every m-subset M with $m_1 \le |M \cap V_1| \le m - m_2$.*

[1] The check was added while writing this paper. Without it, for $\binom{m}{t} > 256$ the byte could name the wrong t-subset, so L was kept on the wrong t-subset and the bookkeeping, not only the scores, was wrong. Four published coverings were reduced by the unchecked build: $C(39, 9, 7, 11)$, $C(30, 10, 10, 19)$, $C(31, 10, 10, 20)$ and $C(32, 10, 10, 20)$; the last three also exceed the flip-cost limit below, which was raised for those runs. Like every covering the search reduced, they were verified exhaustively afterwards (Section 6), so they are valid.

Proof. If $|M \cap V_1| \geq m_1$, an m_1-subset of $M \cap V_1$ meets some B_1 in at least t_1 points; likewise $|M \cap V_2| \geq m_2$ gives B_2. The blocks are disjoint, so $|(B_1 \cup B_2) \cap M| \geq t_1 + t_2 \geq t$. □

An m-subset has $a = |M \cap V_1|$ in $[\mathrm{lo}, \mathrm{hi}] = [\max(0, m - v_2), \min(m, v_1)]$. Products whose intervals $[m_1, m - m_2]$ together contain every feasible a form a (v, k, t, m) covering of size at most the sum of their sizes (duplicate blocks are merged in the explicit covering). A factor with $t_i = 0$ is a single empty block, so $C(v, 0, 0, 0) = 1$. Restricting to $t_1 + t_2 = t$ loses nothing: a term with $t_1 + t_2 > t$ can lower t_2 (or t_1) and, by the third rule of Proposition 7, not grow.

Proposition 6 (Complement duality)**.** *$|\bar{B} \cap \bar{M}| = v - k - m + |B \cap M|$. Hence the complements of a $(v, v - k, t', v - m)$ covering with $t' = \max(0, v - k - m + t)$ form a (v, k, t, m) covering of the same size.*

Proof. $|\bar{B} \cap \bar{M}| = v - |B \cup M| = v - k - m + |B \cap M|$. If $v - k - m + t \geq 0$ the two conditions are equivalent. Otherwise $|B \cap M| \geq k + m - v > t$ for every B and M, and any single block is a covering. □

Proposition 7 (Monotonicity)**.** *$C(v, k, t, m)$ is at most each of $C(v, k, t, m - 1)$; $C(v, k - 1, t, m)$; $C(v, k, t + 1, m)$; $C(v + 1, k, t, m)$; and $C(v - 1, k, t, m - 1)$.*

Proof. An m-set contains an $(m - 1)$-set; a block can be completed; meeting M in $t + 1$ points implies meeting it in t. Deleting the extra point from a covering on $v + 1$ points (and completing the blocks) gives a covering on v points, since every $M \subseteq V$ meets each block in the same points. An m-subset of V has at least $m - 1$ points among the first $v - 1$. □

We call the last two rules (one point more, or one point fewer together with $m-1$) the *one-point rules*.

5.2 The scan

The scan keeps a value $c(v, k, t, m)$ for $v \leq 90$, $k \leq 25$, $t \leq m \leq 20$. It starts from trivial values (1 if $t = 0$ and $m \leq v$; 0 if $m > v$; 1 if $v \leq k$; $\lceil (v - m + 1)/k \rceil$ if $t = 1$) and from the smallest covering available for each entry, in the live table or among our own coverings. Then, for up to six rounds and for each v in increasing order, it applies Proposition 7, the best product (for $2 \leq t \leq k < v$) and complement duality. For the product, fix $v_1 \leq v/2$ and let

$$W(m_1, m_2) = \min_{k_1 + k_2 = k,\ t_1 + t_2 = t} c(v_1, k_1, t_1, m_1)\, c(v_2, k_2, t_2, m_2),$$

made non-increasing in both arguments by running minima (a term with smaller m_i also serves larger ones). The recursion

$$F(\mathrm{lo}) = 0, \qquad F(r + 1) = \min_{\mathrm{lo} \leq l \leq r} \big(F(l) + W(l,\, m - r) \big)$$

finds the cheapest partition of $[\mathrm{lo}, \mathrm{hi}]$ into consecutive intervals $[l, r]$, each covered by one term; since W is non-increasing, no cover by overlapping intervals is cheaper. The value for v_1 is $F(\mathrm{hi} + 1)$. The scan records the rule and v_1 used for each improvement; the construction tree is recovered by recomputing the terms, expanded into explicit blocks, and certified (Section 6).

Example. $C(60, 12, 6, 7)$ had 82,868 blocks in the table. With $v_1 = 16$ the scan found six terms (Table 3), 58,803 blocks in total; the search then reduced the explicit covering to 58,228.

on V_1 ($v_1 = 16$)	on V_2 ($v_2 = 44$)	$a = \lvert M \cap V_1 \rvert$	blocks
$C(16, 12, 6, 7) = 4$	$C(44, 0, 0, 0) = 1$	7	4
$C(16, 10, 5, 5) = 34$	$C(44, 2, 1, 1) = 22$	5–6	748
$C(16, 8, 4, 4) = 30$	$C(44, 4, 2, 3) = 78$	4	2,340
$C(16, 6, 3, 3) = 38$	$C(44, 6, 3, 4) = 387$	3	14,706
$C(16, 4, 2, 2) = 20$	$C(44, 8, 4, 5) = 1{,}175$	2	23,500
$C(16, 0, 0, 0) = 1$	$C(44, 12, 6, 6) = 17{,}505$	0–1	17,505
			58,803

Table 3: The product construction for $C(60, 12, 6, 7)$. Each row covers the 7-subsets M with a in the given range.

Reducing the products. Products cover many m-subsets several times, so the explicit covering is reduced. Lottery products were shrunk by the large-cover search, or, where it refuses the parameters, only stripped of *redundant* blocks (blocks that are the only cover of no m-subset) by the checker of Section 6. Constructed covering designs were only stripped. To decide which published components are worth downloading, the scan is also run as if every table entry were available to us.

6 Verification

Covering designs found by search were checked by an independent script that counts every t-subset.

Lottery coverings found by search were checked exhaustively by a separate counting program, or for some entries by a CP-SAT model whose infeasibility shows that no m-subset is uncovered.

Constructed coverings are certified by construction. A checker rebuilds the block set from the recipe with its own code and asserts the hypothesis of every step (Propositions 5–7); the components are verified separately as above. Every constructed covering that was shrunk or stripped was also checked by a second exhaustive program. It rejects malformed input and removes duplicate blocks, then counts for every m-subset the blocks meeting it in at least t points, in a 16-bit counter per m-subset (2 bytes each, so up to about 10^{10} m-subsets on our machine). A wrapped counter can only report a covered set as uncovered or keep a redundant block, never accept a non-covering. It then deletes redundant blocks one at a time in file order, re-checking each against the current counts; deletions only increase the number of m-subsets each remaining block covers alone, so one pass suffices, and the result is irredundant (not necessarily minimum). According to our batch logs, 399 of the 630 constructed coverings we listed were checked this way at their published size. The other 231 (218 lottery coverings, 95 of them with $\binom{v}{m} > 2 \cdot 10^9$, and 13 covering designs) rest on the construction certificate and the Repository's validator.

Finally, the Covering Repository validates every upload with its own program before accepting it.

7 A random sample of table entries

The published coverings of Section 8 are entries on which a method succeeded, chosen by us; they say neither how often the methods succeed on an arbitrary entry nor whether other methods would have succeeded too. To answer both we ran a comparison whose protocol was committed to the repository before any run (`baselines/random_sample/PROTOCOL.md`). It fixed the population, the sampling, the methods, the budget, the outcomes and three hypotheses, but no statistical tests or decision rules; every test below is therefore exploratory. All runs used one desktop computer (AMD Ryzen 5 5600X, 6 cores and 12 threads; 28 GB of memory available to the programs).

Population and sample. From the Covering Repository tables of 24 October 2026, an entry (v, k, t, m) is eligible if $k < v$, the table value is between 10 and 2,000, $t \geq 3$ for covering designs and $t \geq 2$ for lottery coverings, and $\binom{v}{t} \leq 3 \cdot 10^7$ (covering designs) or $\binom{v}{m} \leq 3 \cdot 10^7$ (lottery coverings), so that every compared program can run it. Entries then held by one of our coverings are excluded, since there the start covering would be our own search output: 9.8% of the otherwise eligible covering-design entries and 1% of the lottery entries. Of the 394 excluded covering-design entries 388 were held by search coverings (almost all from the weighted search) and 6 by constructed ones; of the 99 lottery entries, 62 and 37. Entries the weighted searches had tried without success remain in the population, so the exclusion biases the sample against the weighted searches, by an amount we cannot measure. The population is the small and medium entries held by others; it says nothing about entries with large $\binom{v}{m}$ (Section 9). Entries were stratified by t (3, 4, 5, ≥ 6) for covering designs and by $m - t$ (1, 2, 3, ≥ 4) for lottery coverings, put in random order with a fixed seed, and the first 75 per stratum with an available start covering were taken (none was skipped): 300 covering-design entries and 300 lottery entries, drawn separately. The protocol first fixed 25 per stratum; it was amended to 75 while the first runs were in progress, before any outcome had been looked at. During the runs a concurrent-write bug garbled some result lines; the runner was patched and those runs were repeated from the start (repository history). Stratum sizes in the population differ, so the pooled rates below are rates in the sample; weighting the strata by their population sizes changes them by at most one percentage point.

Methods. Each method started from the table's covering and searched decrementally for 120 seconds of wall time, single-threaded, with three seeds; 11 runs shared the machine. For covering designs the start covering is the archived La Jolla covering, which for 64 entries is larger than the current table value and must first be brought down to it. The methods are ablations within our own framework, not reimplementations of other authors' programs. For covering designs: the incremental search of Section 3.1 (*weighted*; tenure 7, noise 0.05); the same program with the weight increase switched off (*tabu*); and simulated annealing over the same moves in the style of [16] (*annealing*), whose temperature schedule was tuned on a single entry, $C(57, 12, 3)$. For lottery coverings: the search of Section 3.2 with 1,000 candidate blocks and tenure 5 (*weighted*); the same with the weight increase switched off (*tabu*); and direct m-subset scoring with the same settings (*direct*), which is also weighted. An entry counts as improved by a method if its best covering over the three seeds is smaller than the live table value. Each of the 69 improved covering designs and 25 improved lottery coverings (best per entry and method) was checked exhaustively and is valid. None of these coverings is counted in Section 8.

Covering designs. Table 4 gives the improvement rates. The weighted search improved 43 entries, the tabu variant 25 and annealing one. On the 236 entries whose start covering equalled

Stratum	entries	weighted	tabu	annealing / direct	any
covering designs (third method: annealing)					
$t = 3$	75	14	10	0	
$t = 4$	75	15	5	0	
$t = 5$	75	7	6	0	
$t \geq 6$	75	7	4	1	
all	300	43	25	1	54
		14.3%	8.3%	0.3%	18.0%
		(10.8–18.8)	(5.7–12.0)	(0.1–1.9)	(14.1–22.7)
lottery coverings (third method: direct)					
$m - t = 1$	75	2	10	4	
$m - t = 2$	75	1	2	2	
$m - t = 3$	75	1	1	1	
$m - t \geq 4$	75	0	0	1	
all	300	4	13	8	16
		1.3%	4.3%	2.7%	5.3%
		(0.5–3.4)	(2.5–7.3)	(1.4–5.2)	(3.3–8.5)

Table 4: Entries of the random sample improved within 3×120 seconds, by method and stratum, with 95% Wilson intervals for the sample rates. "Any" counts entries improved by at least one method.

the table value the counts are 38, 23 and 1. Of the entries improved by exactly one of weighted and tabu, 29 were weighted-only and 11 tabu-only (exact McNemar test $p = 0.006$; $p = 0.026$ after Holm correction over the six pairwise improvement tests of this section; the sign tests below are not included in it, and none of them is significant). Compared on the median size over seeds, the weighted search was smaller than tabu on 36 entries, equal on 239 and larger on 25 (sign test $p = 0.20$), and smaller than annealing on 47 with no losses. Most entries are ties at the table value, so the median relative gap is 0 for every method. Where the weighted search improved an entry, the gain was small: median 0.71%, mean 1.08%. The advantage of weighting is not uniform. On entries with $k \geq 16$, 26 were weighted-only and 4 tabu-only, and on entries of 50–199 blocks 18 and 0; but on entries with $k < 16$ it was 3 against 7, and on entries of 800 blocks or more 1 against 6. Annealing went below its start size in only 5 of 900 runs, so with this budget and schedule it serves only as a sanity check: the result says that our annealing settings do not work from a table covering in two minutes, not that annealing cannot. The hypothesis stated in the protocol, that the weighted search beats both alternatives on covering designs, holds against annealing, and against tabu on the improvement rate overall but not on median size and not on every kind of entry.

Lottery coverings. Every method improved few lottery entries, and most improvements came from $m - t = 1$. The t-level search improved 13 entries without weighting and 4 with it; 10 entries were tabu-only and 1 weighted-only (McNemar $p = 0.012$, $p = 0.035$ after Holm correction), and on median size tabu was smaller on 8 entries and larger on 2 (sign test $p = 0.11$). Direct scoring, which is weighted, improved 8 entries, between the two: 4 that the weighted t-level search missed and none the other way ($p = 0.12$), and against tabu it was smaller on median size on 3 entries and larger on 8 ($p = 0.23$). So at these settings and this budget, weighting combined with the t-level score estimates (Remark 3) did worst; whether weighting also hurts direct scoring we did not test.

Source	listed	current	median saving	quartiles of saving
covering designs, search	432	401	1.24%	0.67–2.38%
lottery coverings, search	75	65	1.16%	0.43–2.36%
constructed, product only	357		0.67%	0.29–1.70%
constructed, product then reduced	223		1.48%	0.46–6.24%
constructed, all	630	532	0.85%	0.32–2.71%
total	1,137	998		

Table 5: Published coverings on 25 October 2026. The saving of an entry is (table value before our upload − our value) / table value before our upload; quartiles by nearest rank. "Reduced" means shrunk by the search (128 lottery coverings) or only stripped of redundant blocks (95 covering designs). Of the other 50 constructed coverings, 28 use duality or a one-point rule at the top of their tree, and 22 are existing table coverings (ours or others') that were shrunk or stripped.

The protocol expected the t-level methods to beat direct scoring and weighting not to be clearly better than tabu. The second hypothesis held, and the data point to weighting being worse. The first held for tabu, not significantly, and failed for the weighted search. The improvements were mostly by one block (11 of 16 entries), the largest being $C(45, 12, 4, 5)$ from 226 to 210. Our lottery records in Section 8 came mostly from the weighted search, so some of them might also have been found without weighting.

Which entries yield. The following counts for the weighted search on covering designs are descriptive; we fitted no model. Success rose with v (1 of 90 entries for $v < 30$, 17 of 99 for $30 \le v < 50$, 11 of 71 for $50 \le v < 70$, 14 of 40 for $v \ge 70$) and with k (2 of 30 for $k < 10$, 5 of 76 for $10 \le k < 16$, 36 of 194 for $k \ge 16$). It also rose with the ratio of the table value to the Schönheim bound (1 of 32 below 1.1, 23 of 121 at 1.6 or above). By table value it was low at both ends (1 of 91 below 50 blocks, 20 of 90 for 50–199, 20 of 72 for 200–799, 2 of 47 for 800 or more); at the top end two minutes is short for the weighted search, while the tabu variant improved 7 of these 47. Where the start covering was the older La Jolla one and larger than the table value, 5 of 64 entries were improved, against 38 of 236 where it equalled the table value. On lottery coverings, where 16 entries were improved by some method, only 6 of 144 entries with $v < 30$ were improved, and 1 of 153 entries below 50 blocks. Small entries, and lottery entries with small v, seem close to the best these methods reach.

8 Published coverings

This section reports what the methods produced on entries we chose ourselves, so it shows where they succeeded, not how often. Table 5 summarises the published coverings on 25 October 2026. "Listed" counts every covering we uploaded that improved its entry when uploaded; "current" counts those that still hold it. Of the 139 that no longer do, 135 (on 130 distinct entries) are now beaten by other contributors and 4 by our own constructed coverings. "Our value" is our current best for the entry, which for 48 lottery and 9 search covering designs includes a later reduction by the large-cover search.

Table 6 breaks the same coverings down by the combination of methods that produced them.

Kind	Method	listed	current
covering designs	recompute search (first implementation)	106	98
	+ later large-cover shrink	4	4
	incremental search	317	294
	+ later large-cover shrink	5	5
lottery coverings	lottery search	27	20
	+ later large-cover shrink	48	45
constructed covering designs	product	150	147
	product, then redundant blocks removed	95	94
	one-point rule, then redundant blocks removed	5	5
	duality	1	0
	existing covering, redundant blocks removed	21	21
constructed lottery coverings	product	207	118
	product, then large-cover shrink	128	124
	one-point rule, then large-cover shrink	7	7
	duality	15	15
	existing covering, large-cover shrink	1	1
total		1,137	998

Table 6: Published coverings by method on 25 October 2026. "Current" compares our value with the live table.

Covering designs. Of the 432 listed, 110 were first found by the recompute implementation and 322 by the incremental one; 9 were later reduced further by the large-cover search. By t they are $t = 3$: 265, $t = 4$: 112, $t = 5$: 38, $t = 6$: 14, $t = 7$: 3. From the batch logs: one pass of the incremental search over 3,251 table entries improved its start covering on 319 of them, 312 of which improved the table (188 new records and 124 improvements of our own earlier records); a second pass of 600-second runs from our own best coverings on those 319 shrank 249 further, by 1,629 blocks in total. The gains came almost entirely from entries that other contributors had improved in 2025–26; the oldest entries (1996–2005) did not improve.

Lottery coverings. The 75 search records are for $(t, m) = (3, 4)$ (39), $(4, 5)$ (34) and $(4, 6)$ (2); for example $C(78, 9, 3, 4)$ went from 672 to 647, and $C(74, 8, 3, 4)$ from 784 to 765 and later, with the large-cover search, to 756. Among the search settings we tuned (Appendix A), none mattered as much as which entry was attacked; the weighting itself, which we did not vary in these batches, mattered more (Section 7). The following counts are from our batch logs, for entries we chose, and are anecdotal. On records set or re-optimised in 2025–26 the search succeeded on 0 of 102; on older (≤ 2023) records by one prolific contributor or imported from earlier literature with $t = 4$, $m = 5$, $v \geq 40$ it succeeded on 15 of 32. For $(t, m) = (3, 4)$ from the same sources the rate was roughly a tenth for $v \geq 40$ and near zero for $v < 40$ (we did not record the denominators). This is the opposite of what we saw for covering designs, where recently improved entries were the ones that yielded; we take it to depend on how the competing entries were made rather than on their age, but did not test this.

Constructed coverings. 580 of the 630 listed constructed coverings are products at the top of their tree, and 223 of these were reduced afterwards. The gain from reducing varies widely:

$C(56, 14, 7, 8)$ fell from 143,599 in the table to 106,701 by a seven-term product and to 97,928 after shrinking (-8%), and $C(55, 14, 7, 8)$ from 122,870 to 95,732 and then 83,995 (-12%); but $C(60, 6, 5, 7)$, built as 231,388 $\to$ 192,965 by a product with a one-point factor, shrank only to 192,940.

9 Where the methods do not work

Section 7 measured how often the searches succeed on random entries of moderate size. What we observed beyond it:

- *Strong recent records.* In our batches (entries we chose; anecdotal) the lottery search improved none of 102 records set or re-optimised in 2025–26, and the covering-design search did not improve the oldest entries (1996–2005).
- *Large m.* The search needs memory proportional to $\binom{v}{m}$, which rules out most lottery entries (the median entry held by the table's largest contributor has about $7.6{\cdot}10^{13}$ m-subsets). Those would need a method that never enumerates m-subsets, such as lazy constraint generation with a fast exact checker. Exhaustive verification has a similar limit, which is why 231 constructed coverings rest on their construction certificate.
- *Clause weighting with t-level estimates.* On the random sample the t-level search without weighting improved more lottery entries than with it (13 against 4, Section 7), at one setting and one two-minute budget. We have not tested other weighting rules, longer runs, or direct scoring without weighting.
- *Lottery entries in general.* The lottery methods improved 1.3–4.3% of random entries, and only 6 of 225 entries with $m - t \geq 2$ were improved by any method.
- *Large covering designs in short runs.* On covering designs of 800 blocks or more the weighted search improved 2 of 47 random entries in two minutes, fewer than the tabu variant (7).
- *Constructions.* Products rarely beat entries whose value comes from search, and reducing a product can gain anything from almost nothing to 12% (Section 8).
- *Ideas that gave nothing.* A two-for-one block exchange, restarting a level with another deletion, and re-shrinking older coverings by other contributors (Appendix A).

Limitations. All results are upper bounds; we prove no lower bounds and claim no optimality, and the gaps to the Schönheim bound remain large (Table 1). Most timings are single runs, and several figures in Section 8 come from batch logs rather than from the published record lists. The lottery scores are estimates (Remark 3). The random sample covers only entries of moderate size not held by us, with one short budget and one setting per method; longer runs or tuning could change the ranking, and the three methods share our implementation of the framework, so the comparison is of scoring and weighting rules rather than of other authors' programs. We have not measured how often the lottery score estimates change the chosen move.

10 Discussion

On covering designs, clause weighting in a decremental tabu search is the clearest positive result: on a random sample of entries held by others it improved one entry in seven within two minutes,

more than the same search without weighting, mainly on entries with large k and moderate size, and far more than our annealing settings. The gains are small, typically under 1%, and the success rate rises with the ratio of the table value to the Schönheim bound; small entries hardly moved.

The construction scan produced the largest number of records. It contains no new construction: it applies known product, duality and monotonicity rules [6, 14] systematically to a live table that contributors keep improving, and it pays off because a fresh record anywhere in the table can make a product beat many other entries. Reducing the products by search then removes blocks the product covers several times.

On lottery coverings the t-level scoring makes the search practical on large covers, but the random sample does not support the clause weighting there: unweighted, the t-level search improved the most entries, though few in absolute terms. Two directions follow. One is to understand why weighting combined with the score estimates of Remark 3 does worse, by measuring how often the estimates mislead and by testing direct scoring without weighting. The other is to reach the many lottery entries whose $\binom{v}{m}$ is far beyond enumeration, which needs a search that never lists m-subsets.

Data and code availability

The code is at [21] under the MIT licence: `wls128` (recompute) and `wls2` (incremental) for covering designs; `wlsm` (direct m-subset scoring), `wlsm2` (Section 3.2) and `wlsm2big` (Section 4) for lottery coverings; the checkers `verify.py`, `mverify` and `zerocov`; the scan `prodscan`; and `derived/build.py`, which rebuilds and checks any constructed covering; and, in `baselines/random_sample/`, the protocol, sampling script, sample, run script, raw results and analysis script of Section 7. The repository also holds the record lists with previous values and the recipe of every constructed covering. The component coverings are not redistributed, since coverings downloaded from the Covering Repository are for personal use; each is identified by its parameters, size and SHA-1 hash and is available on the Covering Repository, as are the coverings themselves.

Acknowledgements

We thank Giovanni Acerbi for maintaining the Covering Repository and its validator, and the contributors whose published coverings served as starting points and components. The code was written with the help of an AI assistant (Claude, Anthropic); we designed the experiments, checked the results and are responsible for the content.

A Tuning the lottery search

B Control of the large-cover search

These are the settings of our batches; most are options, off by default.

Deletion. The block to delete minimises the weighted cost $\sum_{T\subseteq B,\ \mathrm{ct}(T)=1} L(T)$. Candidates come from a pool of the 2,000 blocks of lowest cost, rebuilt every 500 deletions; pool entries are re-scored in order of their stored cost until the next stored cost is no better than the best found, or a cost of 0 is found.

Change	Effect (our measurements)
t-level scores and global weight counter	10–25 times more steps per second than direct m-subset scoring
score up to 1,000 candidate blocks instead of 12	about twice as many records found
stop an entry after 25 s without shrinking	about 3.4 times more entries per hour, few records lost
three runs per entry (seeds, tabu tenure 2/5/10)	finds records single runs miss
two-for-one block exchange	correct, no extra records
restart a level with another deleted block	no measurable gain

Table 7: Effects measured while tuning the lottery search.

Weights. Weights are kept between levels (by default they are reset to 1), and the bump size δ grows by $\max(1, \lfloor 0.03\,\delta \rfloor)$ per level, which amounts to decaying older weights. When δ exceeds 2^{24}, all weights and δ are reset.

Reverts. If a deletion has not been repaired after 3,000 steps, or 120 seconds since the last full covering or revert, or once the number of uncovered m-subsets exceeds $\max(1000, 50u_0)$ where u_0 is the count just after the deletion, the search returns to the last full covering through an undo log of moves. It then re-scores all blocks with the current weights and deletes the block of rank $r \bmod 64$, where r counts reverts so far.

Stopping and saving. A run stops after 45–60 seconds without a smaller covering, or at 300–600 seconds. Coverings are saved at most every 30 seconds and at exit, including on termination signals; our batch watchdog killed overrunning runs and kept the last saved covering.

References

[1] R. Bertolo, I. Bluskov and H. Hämäläinen, Upper bounds on the general covering number $C_\lambda(v, k, t, m)$, *J. Combin. Des.* 12(5) (2004) 362–380.

[2] J. A. Bate and G. H. J. van Rees, Lotto designs, *J. Combin. Math. Combin. Comput.* 28 (1998) 15–39.

[3] Covering Repository, `https://www.coveringrepository.com`, accessed 25 October 2026.

[4] D. Cushing and D. I. Stewart, Applying constraint programming to minimal lottery designs, *Constraints* 29 (2024) 1–21. doi:10.1007/s10601-024-09368-5.

[5] K. Fadlaoui and P. Galinier, A tabu search algorithm for the covering design problem, *J. Heuristics* 17(6) (2011) 659–674. doi:10.1007/s10732-010-9150-2.

[6] D. M. Gordon, G. Kuperberg and O. Patashnik, New constructions for covering designs, *J. Combin. Des.* 3(4) (1995) 269–284. doi:10.1002/jcd.3180030404.

[7] D. M. Gordon, La Jolla Coverings Repository, version 1.2, Zenodo, 2026. doi:10.5281/zenodo.19735294. See also `https://dmgordon.org/covering-designs/`.

[8] D. M. Gordon and D. R. Stinson, Coverings, in C. J. Colbourn and J. H. Dinitz (eds.), *Handbook of Combinatorial Designs*, 2nd ed., Chapman & Hall/CRC, 2007, Chapter VI.11.

[9] B. P. C. Li, *Some Results on Lotto Designs*, PhD thesis, University of Manitoba, 1999. `http://hdl.handle.net/1993/1660`.

[10] P. C. Li and G. H. J. van Rees, Lotto design tables, *J. Combin. Des.* 10(5) (2002) 335–359. doi:10.1002/jcd.10020.

[11] C. Luo, W. Xing, S. Cai and C. Hu, NuSC: an effective local search algorithm for solving the set cover problem, *IEEE Trans. Cybern.* 54(3) (2024) 1403–1416. doi:10.1109/TCYB.2022.3199147.

[12] F. Margot, Small covering designs by branch-and-cut, *Math. Program.* 94(2–3) (2003) 207–220.

[13] W. H. Mills and R. C. Mullin, Coverings and packings, in J. H. Dinitz and D. R. Stinson (eds.), *Contemporary Design Theory*, Wiley, 1992, 371–399.

[14] F. Montecalvo, Some constructions of general covering designs, *Electron. J. Combin.* 19(3) (2012) P28. arXiv:1205.4994.

[15] P. Morris, The breakout method for escaping from local minima, in *Proc. AAAI-93*, 1993, 40–45.

[16] K. J. Nurmela and P. R. J. Östergård, Upper bounds for covering designs by simulated annealing, *Congr. Numer.* 96 (1993) 93–111.

[17] C. Gao, X. Yao, T. Weise and J. Li, An efficient local search heuristic with row weighting for the unicost set covering problem, *European J. Oper. Res.* 246(3) (2015) 750–761. doi:10.1016/j.ejor.2015.05.038.

[18] A. Balint and U. Schöning, Choosing probability distributions for stochastic local search and the role of make versus break, in *Theory and Applications of Satisfiability Testing – SAT 2012*, Lecture Notes in Computer Science 7317, Springer, 2012, 16–29.

[19] F. Hutter, D. A. D. Tompkins and H. H. Hoos, Scaling and probabilistic smoothing: efficient dynamic local search for SAT, in *Principles and Practice of Constraint Programming (CP 2002)*, LNCS 2470, Springer, 2002, 233–248. doi:10.1007/3-540-46135-3_16.

[20] J. Schönheim, On coverings, *Pacific J. Math.* 14(4) (1964) 1405–1411.

[21] W. van Staden, covering-design-records, `https://github.com/WynandVStaden/covering-design-records`.